PHP - lire, comprendre et écrire du code

Cours - Sessions 1 et 2

BTS SIO 2 - option SLAM

Conditions • Fonctions • Tableaux • foreach • Cybersécurité • Git

Ce document complète le diaporama. Il peut être relu à la maison et sert de référence pour les exercices.

Avant de coder

Programmer ne consiste pas seulement à connaître une syntaxe. Il faut être capable de lire un programme, de comprendre les données utilisées, de prévoir ce qu'il va faire, puis de vérifier son comportement.

Méthode de travail : lire → comprendre → prédire → tester → modifier → expliquer → enregistrer le travail dans Git.

Au début, il est normal de lire plus de code qu'on en écrit. Les notions vues ici reviendront tout au long de l'année : dans les formulaires, l'accès aux bases de données, les contrôles d'accès, Java, JavaScript et les épreuves E6/E7.

1. Conditions et booléens

Une condition est une expression qui peut être évaluée par PHP. Son résultat est un booléen : soit true, soit false. Une condition permet ensuite au programme de prendre une décision.

```
$age = 20;
$estMajeur = $age >= 18;

var_dump($estMajeur);
```

Affichage : `bool(true)`

La variable $estMajeur contient true parce que 20 est bien supérieur ou égal à 18. Une comparaison ne produit donc pas du texte : elle produit une valeur logique.

Le if

if signifie « si ». PHP évalue la condition placée entre parenthèses. Si elle vaut true, le bloc entre accolades est exécuté.

```
$age = 20;

if ($age >= 18) {
    echo "Majeur";
}
```

Affichage : `Majeur`

Structure à retenir : if (condition) { instructions }

Le else et le elseif

else signifie « sinon ». Il permet de prévoir ce qui doit se passer quand la condition du if est fausse. elseif ajoute un cas intermédiaire.

```
$age = 16;

if ($age >= 18) {
    echo "Majeur";
} else {
    echo "Mineur";
}
```

Affichage : `Mineur`

Avec plusieurs cas :

```
$note = 13;

if ($note >= 16) {
    echo "Très bien";
} elseif ($note >= 10) {
    echo "Admis";
} else {
    echo "À revoir";
}
```

Affichage : `Admis`

Comparer des valeurs

En PHP, == compare principalement les valeurs et peut effectuer des conversions de type. === compare la valeur et le type. Dans les exercices, on privilégie généralement les comparaisons strictes === et !==.

```
var_dump(5 == "5");
var_dump(5 === "5");
```

Affichage : `bool(true)`
`bool(false)`

Combiner des conditions

Une règle métier contient souvent plusieurs critères. PHP fournit trois opérateurs très courants :

- ! signifie NON : il inverse un booléen.
- && signifie ET : toutes les conditions reliées doivent être vraies.
- || signifie OU : au moins une des conditions reliées doit être vraie.

```
$role = "admin";
$compteActif = true;

if ($compteActif && ($role === "admin" || $role === "prof")) {
    echo "Accès autorisé";
} else {
    echo "Accès refusé";
}
```

Affichage : `Accès autorisé`

Les parenthèses permettent de rendre la règle plus lisible. Ici, le compte doit être actif ET le rôle doit être admin OU professeur.

Le symbole !

```
$connecte = false;

if (!$connecte) {
    echo "Connexion requise";
}
```

Affichage : `Connexion requise`

!$connecte se lit « l'utilisateur n'est pas connecté ». La variable vaut false ; l'opérateur ! l'inverse donc en true.

Questions de vérification

1. Que vaut $age >= 18 si $age vaut 15 ?
2. Dans quel cas le bloc else est-il exécuté ?
3. Quelle différence entre == et === ?
4. Comment traduire « actif ET admin » en PHP ?

2. Afficher et construire une réponse

Les exemples du cours utilisent souvent echo. echo envoie du texte ou une valeur vers la sortie du script. Dans un navigateur, cette sortie apparaît dans la page générée par PHP.

Concaténer avec le point

Concaténer consiste à assembler plusieurs morceaux pour construire une seule chaîne de caractères. En PHP, on utilise le point .

```
$nom = "Nora";
$role = "gestionnaire";

echo "Bonjour " . $nom . ", rôle : " . $role;
```

Affichage : `Bonjour Nora, rôle : gestionnaire`

L'opérateur ternaire

Le ternaire est une écriture courte pour choisir entre deux valeurs. Sa forme est : condition ? valeur_si_vrai : valeur_si_faux. Il est pratique pour un choix simple, mais un if classique reste préférable si la logique devient longue.

```
$stock = 0;

echo $stock > 0 ? "Disponible" : "Indisponible";
```

Affichage : `Indisponible`

On peut aussi l'utiliser avec une fonction qui retourne un booléen :

```
echo estMajeur(20) ? "oui" : "non";
```

Lecture : si estMajeur(20) retourne true, afficher « oui » ; sinon afficher « non ».

3. Fonctions : regrouper et réutiliser une logique

Une fonction est un bloc de code auquel on donne un nom. Elle permet d'éviter de recopier la même logique à plusieurs endroits. Elle peut recevoir des données en entrée, appelées paramètres, puis renvoyer un résultat avec return.

```
function estMajeur(int $age): bool
{
    return $age >= 18;
}
```

Dans cet exemple, $age est un paramètre de type int. Le : bool indique que la fonction doit renvoyer un booléen.

Appeler une fonction

Définir une fonction ne l'exécute pas. Pour l'utiliser, on écrit son nom suivi de parenthèses et des valeurs que l'on souhaite lui transmettre.

```
$resultat = estMajeur(20);
var_dump($resultat);
```

Affichage : `bool(true)`

La valeur 20 est transmise au paramètre $age. La fonction effectue la comparaison puis renvoie true. Le résultat peut être stocké, testé, affiché ou transmis à une autre fonction.

Fonction et procédure

En PHP, le mot-clé function sert dans les deux cas. On utilise souvent le mot « procédure » pour parler d'un bloc qui réalise une action mais ne renvoie pas de résultat utile. Le type void signifie précisément qu'aucune valeur n'est retournée.

```
function afficherBienvenue(string $nom): void
{
    echo "Bonjour " . $nom;
}

function calculerTtc(float $prix): float
{
    return $prix * 1.2;
}

afficherBienvenue("Nora");
echo calculerTtc(10);
```

Affichage :
```
Bonjour Nora
12
```

À retenir : void = pas de valeur renvoyée. bool, int, float, string ou array indiquent le type de valeur attendu en retour.

return arrête la fonction

Dès qu'un return est exécuté, la fonction s'arrête et rend la valeur indiquée à l'endroit où elle a été appelée. Les instructions situées plus bas dans cette fonction ne sont alors plus exécutées pour ce passage.

Clauses de garde

Une clause de garde est un test placé au début d'une fonction pour éliminer immédiatement un cas impossible ou interdit. Cette écriture évite d'imbriquer de nombreux if et garde la règle principale plus lisible.

```php
function peutCommander(bool $compteValide, int $stock): bool
{
    if (!$compteValide) {
        return false;
    }

    if ($stock <= 0) {
        return false;
    }

    return true;
}

var_dump(peutCommander(true, 3));
```

Affichage : `bool(true)`

La fonction refuse d'abord un compte invalide, puis un stock nul ou négatif. Si elle arrive au dernier return, les deux conditions indispensables sont respectées.

Une fonction peut en appeler une autre

Une fonction devient vraiment utile lorsqu'elle peut être réutilisée dans une autre. Cela évite de recopier la règle métier.

```php
function produitDisponible(array $produit): bool
{
    return $produit["stock"] > 0;
}

function compterProduitsDisponibles(array $produits): int
{
    $compteur = 0;

    foreach ($produits as $produit) {
        if (produitDisponible($produit)) {
            $compteur++;
        }
    }

    return $compteur;
}
```

compterProduitsDisponibles() ne recopie pas la condition sur le stock : elle demande simplement à produitDisponible() de répondre pour chaque produit.

Questions de vérification

5. Quelle différence entre définir et appeler une fonction ?
6. Que signifie : void ?
7. Que fait return ?
8. Pourquoi utiliser une clause de garde ?
9. Quel intérêt y a-t-il à appeler une fonction depuis une autre ?

4. Tableaux : organiser plusieurs données

Un tableau PHP peut contenir plusieurs valeurs. Il peut être indexé, avec des positions numériques, ou associatif, avec des clés textuelles.

Tableau indexé

```
$notes = array(12, 18, 14);

echo $notes[0];
```

Affichage : `12`

Le premier élément porte l'index 0.

Tableau associatif

```
$produit = array(
    "nom" => "Clavier",
    "stock" => 4
);

echo $produit["nom"];
```

Affichage : `Clavier`

La clé "nom" permet d'accéder directement à la valeur correspondante. Les clés décrivent le sens des données et rendent le code plus lisible.

Une collection de tableaux

On peut aussi placer plusieurs tableaux associatifs dans un tableau indexé. C'est une structure très courante pour représenter une liste de produits, de demandes, de tickets ou d'utilisateurs.

```
$produits = array(
    array("nom" => "Clavier", "stock" => 4),
    array("nom" => "Souris", "stock" => 0)
);
```

5. foreach : parcourir une collection

foreach traite les éléments d'un tableau un par un. Une convention simple aide à lire le code : le nom de la collection est souvent au pluriel, et le nom de l'élément courant au singulier.

```
foreach ($produits as $produit) {
    echo $produit["nom"] . "<br>";
}
```

Affichage :
```
Clavier
Souris
```

Lecture : $produits représente toute la collection ; $produit représente un seul élément pendant un tour de boucle.

Compter avec foreach

```
$compteur = 0;

foreach ($produits as $produit) {
    if ($produit["stock"] > 0) {
        $compteur++;
    }
}

echo $compteur;
```

Affichage : `1`

$compteur++ ajoute 1 à la valeur du compteur. On l'utilise ici uniquement lorsque le produit possède du stock.

6. Inspecter une variable : echo, print_r() et var_dump()

Ces trois outils ne servent pas exactement au même usage.

Commande	À quoi elle sert
`echo`	Afficher une valeur simple ou du texte.
`print_r($variable)`	Afficher de façon lisible le contenu d'un tableau ou d'une structure.
`var_dump($variable)`	Afficher la valeur avec son type ; très utile pour true, false, int, string, null ou array.

```
$produit = array(
    "nom" => "Clavier",
    "stock" => 4
);

print_r($produit);
var_dump($produit["stock"]);
```

Affichage :
```
Array ( [nom] => Clavier [stock] => 4 )
int(4)
```

Pendant les exercices, var_dump() est particulièrement utile pour vérifier qu'une fonction renvoie bien le type attendu.

7. Rechercher une valeur et gérer l'absence de résultat

Une recherche parcourt souvent une liste jusqu'à trouver l'élément demandé. Si l'élément est trouvé, on peut le retourner immédiatement. En revanche, on ne peut déclarer qu'il est absent qu'après avoir parcouru toute la liste.

```
function rechercherProduit(array $produits, string $nom): ?array
{
    foreach ($produits as $produit) {
        if ($produit["nom"] === $nom) {
            return $produit;
        }
    }

    return null;
}
```

?array signifie que la fonction peut renvoyer un tableau ou null. null représente ici l'absence de résultat.

Point important : return null est placé après le foreach. Au premier élément qui ne correspond pas, il reste encore d'autres éléments à vérifier.

8. Premiers repères de cybersécurité

Les conditions et les fonctions ne servent pas seulement à calculer. Elles peuvent aussi appliquer des règles de sécurité. Un contrôle d'accès consiste à vérifier qui est l'utilisateur et ce qu'il a le droit de faire.

Authentification et autorisation

- Authentification : vérifier l'identité d'un utilisateur (par exemple après une connexion).
- Autorisation : vérifier qu'un utilisateur authentifié possède le droit d'effectuer une action.
- Traçabilité : conserver des éléments permettant de savoir qui a effectué une action importante, quand et sur quelle ressource.

```
$connecte = true;
$role = "utilisateur";

if (!$connecte) {
    echo "Connexion obligatoire";
} elseif ($role !== "admin") {
    echo "Accès refusé";
} else {
    echo "Administration";
}
```

Affichage : `Accès refusé`

L'utilisateur est bien authentifié puisqu'il est connecté, mais il n'est pas autorisé à accéder à la partie administration.

Le principe du moindre privilège

Un utilisateur ne devrait disposer que des droits nécessaires à son travail. Donner des droits trop larges augmente les conséquences possibles d'une erreur, d'un compte compromis ou d'une mauvaise manipulation.

À retenir pour la suite : un contrôle sensible doit être réalisé côté serveur. Masquer un bouton dans une page ne suffit pas à empêcher une action interdite.

Les traces peuvent être sensibles

Les journaux techniques peuvent contenir des identifiants, des adresses IP, des chemins de fichiers, des erreurs ou des informations sur l'infrastructure. Ils sont utiles pour la traçabilité, mais ils doivent eux-mêmes être protégés.

Questions de vérification

10. Quelle différence entre authentification et autorisation ?
11. Pourquoi un compte partagé dégrade-t-il la traçabilité ?
12. Pourquoi un simple if peut-il avoir un impact sur la sécurité d'une application ?

9. Utiliser l'IA sans perdre la compréhension

L'IA peut être utile pour relire, tester ou améliorer du code. Elle devient problématique si elle remplace la première réflexion. Le cours utilise trois niveaux simples.

Mode rouge - sans IA : lire, prédire, faire une première tentative et noter les difficultés.

Mode orange - relecture : après une première tentative, demander une vérification de la logique et des cas de test sans demander une solution complète.

```
Je suis étudiant en BTS SIO SLAM.

J’ai d’abord essayé seul.
Voici la consigne :
[coller la consigne]

Voici mon code :
[coller mon code]

Tu es uniquement mon relecteur.
Ne refais pas l’exercice à ma place.
Vérifie la logique, signale les erreurs et propose des cas de test.
```

Mode vert - usage professionnel : sur un code déjà compris et testé, demander des tests supplémentaires, une amélioration de lisibilité ou une courte documentation.

```
Je suis étudiant en BTS SIO SLAM.

Voici mon code fonctionnel :
[coller le code]

Aide-moi à l’améliorer sans changer le résultat.
Propose :
1. des tests supplémentaires ;
2. une amélioration de lisibilité ;
3. une courte documentation.
Explique chaque proposition simplement.
```

Sécurité : ne pas transmettre de vrais mots de passe, tokens, clés privées, fichiers .env ou journaux réels contenant des données sensibles.

10. Git et GitHub

Git enregistre l’historique d’un projet sur l’ordinateur. GitHub héberge un dépôt distant et facilite le partage. Les deux sont liés, mais ce ne sont pas la même chose.

Vocabulaire : un dépôt local se trouve sur votre ordinateur. Un dépôt distant se trouve sur un serveur, par exemple GitHub. origin est le nom conventionnel donné au dépôt distant principal.

Le rituel de fin de séance

```
git status
git add .
git commit -m "Travail de la séance"
git push
```

Commande	À quoi elle sert
`git status`	Voir la branche courante et les fichiers modifiés, ajoutés ou non suivis.
`git add .`	Préparer les modifications pour le prochain commit.
`git commit -m "..."`	Créer un point d’historique local avec un message clair.
`git push`	Envoyer les commits locaux vers le dépôt distant configuré.

Premier envoi vers un dépôt GitHub vide

Si le projet existe déjà sur votre ordinateur et que le dépôt GitHub vient d'être créé vide, on peut initialiser Git localement puis relier les deux.

```
git init
git status
git add .
git commit -m "Initialisation du projet"
git branch -M main
git remote add origin https://github.com/PSEUDO/slam-2026-2027.git
git push -u origin main
```

git branch -M main renomme la branche courante en main. git remote add origin ... ajoute l'adresse du dépôt distant. Le -u de la première commande push mémorise le lien entre la branche locale main et origin/main.

Attention : si le dépôt GitHub contient déjà un README ou d'autres commits, il est souvent plus simple de le cloner plutôt que de créer séparément deux historiques.

Cloner un dépôt existant

clone sert lorsqu'un dépôt existe déjà à distance et que l'on souhaite en créer une copie locale complète.

```
git clone https://github.com/PSEUDO/slam-2026-2027.git
cd slam-2026-2027
git status
```

Vérifier ou changer le dépôt distant

```
git remote -v
```

Cette commande affiche les adresses utilisées pour fetch et push. Si origin pointe vers le mauvais dépôt, deux méthodes sont possibles.

Méthode directe :

```
git remote set-url origin https://github.com/PSEUDO/NOUVEAU_DEPOT.git
```

Ou suppression puis recréation :

```
git remote remove origin
git remote add origin https://github.com/PSEUDO/NOUVEAU_DEPOT.git
```

origin n'est pas une branche : c'est simplement un nom donné à un dépôt distant.

Branches et branche par défaut

Une branche représente une ligne de travail. Elle permet d'isoler une modification sans travailler directement sur la branche principale.

```
git branch
git switch -c test-formulaire
git switch main
```

git switch -c crée une nouvelle branche et bascule dessus. git switch main revient sur main.

La branche par défaut (default branch) est la branche présentée par défaut sur GitHub et utilisée comme référence pour de nombreuses opérations. Elle s'appelle souvent main. Renommer une branche locale avec git branch -M main ne modifie pas à lui seul tous les réglages du dépôt distant.

Publier une nouvelle branche sur GitHub :

```
git push -u origin test-formulaire
```

Récupérer les modifications : git pull

git pull récupère les changements de la branche distante suivie et les intègre dans la branche locale courante. Avant de reprendre un projet partagé ou utilisé sur plusieurs ordinateurs, c'est un bon réflexe de vérifier l'état puis de récupérer les nouveautés.

```
git status
git pull

# puis je travaille
git add .
git commit -m "Travail de la séance"
git push
```

En cas de doute : commencer par git status et git remote -v. Ces deux commandes expliquent souvent une grande partie des problèmes Git courants.

11. Ce qu'il faut savoir expliquer

- Ce qu'est une condition et pourquoi elle produit true ou false.
- La différence entre if, else et elseif.
- Le rôle de !, &&, ||, === et !==.
- Ce qu'est une fonction, comment on l'appelle et ce que fait return.
- La différence pratique entre une fonction qui retourne une valeur et une procédure void.
- L'intérêt d'une clause de garde.
- La différence entre un tableau indexé et un tableau associatif.
- Le rôle de foreach et la différence entre une collection et l'élément courant.
- La différence entre echo, print_r() et var_dump().
- Pourquoi return null se place après une boucle de recherche.
- La différence entre authentification, autorisation et traçabilité.
- Le cycle Git : status → add → commit → push, ainsi que clone, pull, remote et branches.

Relecture conseillée entre deux séances

Une courte reprise régulière est plus efficace qu'une longue révision tardive. En une vingtaine de minutes : relire une section, exécuter un exemple sans le modifier, puis le refaire avec une valeur différente et expliquer à voix haute ce qui change.

Objectif : être capable de comprendre et d'expliquer le code, pas seulement de le faire fonctionner.